Ifeakandum AI Integration

Backend Improvement Guide

Prepared for Agent-Assisted Refactoring | June 2025

Overview

This document describes 10 targeted improvements for the Ifeakandum AI Medical Records Analysis System backend. Each issue includes the affected file, a description of the problem, the required fix, and code samples where relevant.

Issues are grouped into three priority tiers:

- 🔴 CRITICAL — Will cause the app to crash or produce incorrect results at runtime.
- 🟡 ARCHITECTURE — Structural problems that affect reliability, scalability, or correctness.
- 🟢 CODE QUALITY — Improvements to maintainability, safety, and readability.

Issue Summary

#	P	Issue	File(s)
1	🔴	Env variable mismatch — client crashes on startup	`src/services/patient_service.py`
2	🔴	asyncio.create_task in sync context — use BackgroundTasks	`src/main.py`
3	🔴	Celery shared_task has no configured broker	`src/tasks.py + celery_backend.py`
4	🟡	Race condition on batch progress counter	`src/tasks.py`
5	🟡	CORS allow_origins=* with credentials=True is invalid	`src/main.py`
6	🟡	Unused schema classes — drift between schema and models	`src/schema/patient_schema.py`
7	🟡	Celery task re-mapping schema is fragile	`src/tasks.py`
8	🟢	load_dotenv() called in multiple files	`src/main.py + patient_service.py`

9	🟢	Hardcoded medication fallback list	`src/services/patient_service.py`
10	🟢	No pagination on list endpoints	`src/main.py`

Critical Issues

Issue 1: Env Variable Name Mismatch

Priority	🔴 CRITICAL
File	`src/services/patient_service.py`
Problem	The client reads os.getenv('DEEPSEEK_API_KEY') but the README and .env.example use OPENROUTER_API_KEY. When the wrong variable is undefined, DeepSeekClient.__init__ raises ValueError immediately — crashing the app on startup before any request is served.
Fix	Align the env variable names across all files. Pick one name (OPENROUTER_API_KEY is more accurate since you are hitting OpenRouter, not DeepSeek directly) and use it everywhere: patient_service.py, .env, and .env.example. Also add a startup check in lifespan() that logs a clear error if the key is missing.

Code	`# patient_service.py — change:`
	`api_key = os.getenv('DEEPSEEK_API_KEY')  # WRONG`
	
	`# to:`
	`api_key = os.getenv('OPENROUTER_API_KEY') # CORRECT`
	
	`# main.py lifespan — add a check:`
	`if not os.getenv('OPENROUTER_API_KEY'):`
	`  raise RuntimeError('OPENROUTER_API_KEY is not set. App cannot start.')`

Issue 2: Use BackgroundTasks Instead of asyncio.create_task

Priority	🔴 CRITICAL
File	`src/main.py — upload_and_analyze_csv endpoint`
Problem	asyncio.create_task() requires an actively running event loop and can fail silently if the loop is not in the expected state. This is not the recommended FastAPI pattern for background work.
Fix	Replace asyncio.create_task with FastAPI's built-in BackgroundTasks. Inject it as a dependency parameter on the route handler.

Code	`# BEFORE:`
	`async def upload_and_analyze_csv(file: UploadFile = File(...)):`
	`  asyncio.create_task(process_csv_in_chunks(content, batch_id))`

	`# AFTER:`
	`from fastapi import BackgroundTasks`
	
	`async def upload_and_analyze_csv(`
	`file: UploadFile = File(...),`
	`background_tasks: BackgroundTasks = BackgroundTasks()`
	`):`
	`background_tasks.add_task(process_csv_in_chunks, content, batch_id)`

Issue 3: Celery Broker Not Configured

Priority	🔴 CRITICAL
File	`src/tasks.py + src/celery_backend.py`
Problem	tasks.py uses @shared_task but celery_backend.py is never imported or wired up. There is no Celery app instance with a broker (Redis/RabbitMQ). Calling .delay() will raise a RuntimeError at runtime because no worker is registered.
Fix	Create a proper Celery app instance in celery_backend.py with a broker URL, then import it in tasks.py. Ensure the app is initialized before any task is decorated.

Code	`# celery_backend.py — define the app:`
	`from celery import Celery`
	`import os`
	
	`celery_app = Celery(`
	`'ifeakandum',`
	`broker=os.getenv('CELERY_BROKER_URL', 'redis://localhost:6379/0'),`
	`backend=os.getenv('CELERY_RESULT_BACKEND',`
	`'redis://localhost:6379/0')`
	`)`
	
	`# tasks.py — replace @shared_task with:`
	`from src.celery_backend import celery_app`
	
	`@celery_app.task(bind=True, max_retries=3)`
	`def analyze_medical_record_task(self, record_id: int, batch_id: str =`
	`None):`
	`...`

Architecture Issues

Issue 4: Race Condition on Batch Progress Counter

Priority	🟡 ARCHITECTURE
File	`src/tasks.py`
Problem	Multiple Celery workers each do: batch.processed_records += 1. This is a read-modify-write operation. Two workers finishing simultaneously will both read the same value, both increment it, and one update will be lost — meaning the

	counter will never reach total_records and the batch will never be marked 'completed'.
Fix	Use a SQL-level atomic increment instead of a Python-level read-modify-write. With SQLModel/SQLAlchemy this can be done with a direct UPDATE statement.

Code	`from sqlalchemy import update`
	
	`# BEFORE (race condition):`
	`batch.processed_records += 1`
	`update_session.add(batch)`
	`update_session.commit()`
	
	`# AFTER (atomic):`
	`update_session.exec(`
	`  update(BatchStatus)`
	`  .where(BatchStatus.batch_id == batch_id)`
	`  .values(processed_records=BatchStatus.processed_records + 1)`
	`)`
	`update_session.commit()`

Issue 5: Invalid CORS Configuration

Priority	🟡 **ARCHITECTURE**
File	`src/main.py`
Problem	Setting allow_origins=["*"] and allow_credentials=True simultaneously is rejected by browsers (CORS spec disallows wildcard origins with credentials). Any frontend using cookies or Authorization headers will receive a CORS error.
Fix	Replace the wildcard with explicit origins. If you do not yet have a production domain, use localhost for development and add real domains before deployment.

Code	`# BEFORE:`
	`allow_origins=['*'],`
	`allow_credentials=True,`
	
	`# AFTER:`
	`allow_origins=[`
	`  'http://localhost:3000',`
	`  'http://localhost:5173',`
	`  'https://your-frontend-domain.com',`
	`],`
	`allow_credentials=True,`

Issue 6: Unused Schema Classes — Drift Risk

Priority	🟡 **ARCHITECTURE**
File	`src/schema/patient_schema.py`
Problem	BatchAnalysisRequest and CSVAnalysisResult are defined in the schema file but never used anywhere. The BatchStatus SQLModel in models.py serves the same

	purpose. These dead classes create confusion about which structure is the source of truth and will inevitably drift from the real models.
Fix	Remove BatchAnalysisRequest and CSVAnalysisResult from patient_schema.py. Confirm all batch-status responses in main.py use the BatchStatus SQLModel directly.

Code	`# DELETE these two classes from patient_schema.py:`
	
	`class BatchAnalysisRequest(BaseModel):  # REMOVE`
	`    ...`
	
	`class CSVAnalysisResult(BaseModel):     # REMOVE`
	`    ...`

Issue 7: Celery Task Re-maps Schema — Fragile Duplication

Priority	🟡 **ARCHITECTURE**
File	`src/tasks.py`
Problem	The Celery task fetches a MedicalRecord from DB and then manually reconstructs a MedicalRecords Pydantic schema object to pass back into deepseek_client.analyze_medical_record(). This duplicates the field-mapping logic already in batch_analysis.py and will break silently if the models or schema fields ever change.
Fix	Refactor DeepSeekClient.analyze_medical_record to accept a plain dict instead of a Pydantic schema. Both the route handler and the Celery task can build and pass a dict without schema reconstruction.

Code	`# patient_service.py — change signature:`
	`async def analyze_medical_record(self, record_data: dict):`
	`    prompt = self._create_analysis_prompt_from_dict(record_data)`
	
	`# tasks.py — simplify to:`
	`record_data = {`
	`    'patient_id': record.patient.patient_id,`
	`    'age': record.patient.age,`
	`    'symptoms': record.primary_symptoms,`
	`    # ...etc`
	`}`
	`ai_analysis = loop.run_until_complete(`
	`    deepseek_client.analyze_medical_record(record_data)`
	`)`

Code Quality Issues

Issue 8: load_dotenv() Called in Multiple Files

Priority	🟢 **CODE QUALITY**

File	`src/main.py + src/services/patient_service.py`
Problem	load_dotenv() is called at the top of both main.py and patient_service.py. While not harmful, it is unnecessary and misleading — it implies each file independently manages its environment, which is not how a FastAPI app should work.
Fix	Call load_dotenv() exactly once, at the entry point (main.py), before any other imports that depend on env vars. Remove it from patient_service.py and any other module.

Code	`# main.py — keep this ONE call, at the very top:`
	`from dotenv import load_dotenv`
	`load_dotenv()`
	
	`# patient_service.py — REMOVE:`
	`from dotenv import load_dotenv  # DELETE`
	`load_dotenv()  # DELETE`

Issue 9: Hardcoded Medication Fallback List

Priority	🟢 **CODE QUALITY**
File	`src/services/patient_service.py — _extract_structured_data()`
Problem	The fallback parser does a keyword scan of the raw AI text against a hardcoded list of medication names (nitroglycerin, labetalol, metoprolol, etc.). This will produce false positive medication recommendations any time those words appear in an explanation or contraindication section — not just as recommended meds.
Fix	Remove the hardcoded medication scan from the fallback. Instead, return the raw AI text in treatment_notes and flag confidence_level as 'low' to signal that manual review is required. Never guess medications.

Code	`# REMOVE the common_meds scan block entirely:`
	`common_meds = ['nitroglycerin', ...]  # DELETE`
	
	`# REPLACE with a clear low-confidence fallback:`
	`if not response['recommended_medications']:`
	`  response['confidence_level'] = 'low'`
	`  response['treatment_notes'] = (`
	`  'AI response could not be parsed into structured recommendations.`
	`'`
	`  'Manual review required. Raw response stored in notes.'`
	`  )`

Issue 10: No Pagination on List Endpoints

Priority	🟢 **CODE QUALITY**
File	`src/main.py — GET /analysis-result and GET /doctor-reports`
Problem	Both GET /api/v1/analysis-result/{patient_id} and GET /api/v1/doctor-reports/{patient_id} perform unbounded select() queries. A patient

	with many records will load everything into memory at once, which will degrade performance and potentially exhaust memory under load.
Fix	Add limit and offset query parameters to both endpoints with sensible defaults (limit=20, max=100).

Code	`from fastapi import Query`
	
	`@app.get('/api/v1/analysis-result/{patient_id}')`
	`async def get_analysis_result(`
	`  patient_id: str,`
	`  limit: int = Query(default=20, le=100),`
	`  offset: int = Query(default=0, ge=0),`
	`  session: Session = Depends(get_session)`
	`):`
	`  statement = (`
	`  select(Analysis)`
	`  .join(MedicalRecord)`
	`  .where(MedicalRecord.patient_id == patient_id)`
	`  .offset(offset).limit(limit)`
	`  )`

Recommended Implementation Order

Apply fixes in this sequence to avoid breaking changes mid-refactor:

Step 1 — Fix env var mismatch (Issue 1)
Align OPENROUTER_API_KEY in patient_service.py and .env.example. Verify the app starts cleanly.

Step 2 — Wire up Celery (Issue 3)
Configure celery_backend.py with a real broker, then update tasks.py to use it. Start a Redis instance locally and confirm a test task executes.

Step 3 — Replace asyncio.create_task (Issue 2)
Swap in BackgroundTasks on the upload endpoint. This is a small change and can be done alongside Step 2.

Step 4 — Fix race condition (Issue 4) and CORS (Issue 5)
Apply the atomic SQL increment and restrict CORS origins. These are independent and can be done together.

Step 5 — Refactor schema/service coupling (Issues 6 and 7)

Remove unused schema classes and refactor the Celery task to use a plain dict. Test both the single-record and batch endpoints after this change.

Step 6 — Code quality cleanup (Issues 8, 9, 10)
Clean up load_dotenv, remove the medication fallback scan, and add pagination. These are low-risk changes that can be batched in a single commit.

Required Environment Variables

Ensure your .env file contains the following variables after completing the above fixes:

```
# API Keys
OPENROUTER_API_KEY=your_openrouter_key_here

# Database
DATABASE_URL=sqlite:///./medical_analysis.db

# Celery (Redis required)
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/0
```